307 lines (252 loc) · 10.8 KB

```python
import math
import os
import random
import sys
import pygame as pg

# 【条件】実行ファイルのあるディレクトリをカレントディレクトリに設定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# 定数設定
WIDTH = 800
HEIGHT = 600
GROUND_Y = 500

# 色定義
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
SKY = (120, 200, 255)
YELLOW = (255, 220, 0)

def check_bound_horizontal(obj_rct: pg.Rect) -> bool:
    """
    オブジェクトが画面の左端より外に出たかを判定する
    """
    if obj_rct.right < 0:
        return True
    return False

class Bird(pg.sprite.Sprite):
    """
    プレイヤー（キャラクター）に関するクラス
    """
    def __init__(self, xy: tuple[int, int]):
        super().__init__()

        # キャラクター画像の読み込みとサイズ調整
        self.img_run = pg.transform.scale(pg.image.load("fig/2.png").convert_alpha(), (60, 80))   # 動いている
        self.img_jump = pg.transform.scale(pg.image.load("fig/6.png").convert_alpha(), (60, 80))  # ジャンプ時

        # ジャンプ時の効果音を読み込む
        self.se_jump = pg.mixer.Sound("fig/sound/junp.wav")

```

```python
        self.image = self.img_run
        self.rect = self.image.get_rect()
        self.rect.center = xy

        self.y_speed = 0
        self.on_ground = True
        self.jump_force = -18
        self.gravity = 1

    def update(self, key_lst: list[bool]):
        # 重力処理
        self.y_speed += self.gravity
        self.rect.y += self.y_speed

        # 地面判定
        if self.rect.bottom >= GROUND_Y:
            self.rect.bottom = GROUND_Y
            self.y_speed = 0
            self.on_ground = True
        else:
            self.on_ground = False

        # 状態に合わせて画像を切り替える
        if self.on_ground:
            self.image = self.img_run
        else:
            self.image = self.img_jump

        # ジャンプ入力
        if key_lst[pg.K_SPACE] and self.on_ground:
            # ジャンプ時に効果音を鳴らす
            self.se_jump.play()
            self.y_speed = self.jump_force
            self.on_ground = False

class Obstacle(pg.sprite.Sprite):
    """
    障害物（赤い矩形）に関するクラス
    """
    def __init__(self, speed):
        super().__init__()
        height = random.randint(40, 80)
        self.image = pg.Surface((50, height))
        self.image.fill(RED)
        self.rect = self.image.get_rect()
        self.rect.left = WIDTH
        self.rect.bottom = GROUND_Y
        self.speed = speed

    def update(self):
        self.rect.x -= self.speed
        if check_bound_horizontal(self.rect):
            self.kill()

class Coin(pg.sprite.Sprite):
    """
    コイン（画像）に関するクラス
    """
    def __init__(self, speed, coin_img):
```

```
        super().__init__()
```



```

    def update(self):
        self.rect.x -= self.speed
        if check_bound_horizontal(self.rect):
            self.kill()

class Score:
    """
    スコアとコイン数を表示するクラス
    """
    def __init__(self):
        self.font = pg.font.SysFont(None, 40)
        self.score_value = 0
        self.coin_value = 0

    def update(self, screen: pg.Surface):
        score_img = self.font.render(f"Score : {self.score_value}", True, WHITE)
        coin_img = self.font.render(f"Coins : {self.coin_value}", True, WHITE)
        screen.blit(score_img, (20, 20))
        screen.blit(coin_img, (20, 60))

def draw_text_with_shadow(screen, text, font, text_color, shadow_color, x, y):
    """文字に影をつけて見やすく描画する補助関数"""
    shadow = font.render(text, True, shadow_color)
    main_text = font.render(text, True, text_color)
    screen.blit(shadow, (x + 3, y + 3)) # 影を右下にずらす
    screen.blit(main_text, (x, y))

def main():
    pg.display.set_caption("Kokaton Run")
    screen = pg.display.set_mode((WIDTH, HEIGHT))
    clock = pg.time.Clock()

    # --- 画像の読み込みと準備 ---

    # 1. 通常背景 (back_graund.jpg)
    bg_img = pg.image.load("fig/back_graund.jpg").convert()
    bg_w = int(bg_img.get_width() * (HEIGHT / bg_img.get_height()))
    bg_img = pg.transform.scale(bg_img, (bg_w, HEIGHT))
    # 元の暗いオーバーレイ（通常背景用）
    dark_overlay = pg.Surface((bg_w, HEIGHT), pg.SRCALPHA)
    dark_overlay.fill((0, 0, 0, 120))
    bg_img.blit(dark_overlay, (0, 0))

    # 2. ゲームオーバー用背景 (died_kokaton.png)
    bg_img_go = pg.transform.scale(pg.image.load("fig/died_kokaton.png").convert_alpha(), (WIDTH, HEIGHT))

    # --- 変更: ゲームオーバー背景を少し明るくするためにアルファ値を 150 から 80 に変更 ---
    bg_img_go_overlay = pg.Surface((WIDTH, HEIGHT), pg.SRCALPHA)
    bg_img_go_overlay.fill((0, 0, 0, 80))  # 80に下げることで背景画像をより明るくはっきり見せます
    bg_img_go.blit(bg_img_go_overlay, (0, 0))
    # --------------------------------------------------------------------------------

```

```python
    # 地面画像の読み込みとサイズ調整
    ground_tile = pg.image.load("fig/jimen.jpg").convert()
    ground_size = HEIGHT - GROUND_Y
    ground_tile = pg.transform.scale(ground_tile, (ground_size, ground_size))

    raw_coin_img = pg.image.load("fig/coin.png").convert_alpha()
    coin_img = pg.transform.scale(raw_coin_img, (35, 35))

    # 効果音を読み込む
    se_coin = pg.mixer.Sound("fig/sound/coin_get.wav")
    se_gameover = pg.mixer.Sound("fig/sound/game_over.mp3")

    # スタート画面用のBGMを読み込んでループ再生
    pg.mixer.music.load("fig/sound/start_bgm.mp3")
    pg.mixer.music.play(-1)

    bird = Bird((150, GROUND_Y - 40))
    obstacles = pg.sprite.Group()
    coins = pg.sprite.Group()
    score = Score()

    is_started = False
    game_over = False
    tmr = 0

    title_font = pg.font.SysFont(None, 100)
    msg_font = pg.font.SysFont(None, 50)

    go_title_font = pg.font.SysFont(None, 120)
    go_score_font = pg.font.SysFont(None, 60)

    while True:
        key_lst = pg.key.get_pressed()

        # イベント処理
        for event in pg.event.get():
            if event.type == pg.QUIT:
                return
            if event.type == pg.KEYDOWN:
                if not is_started:
                    if event.key == pg.K_SPACE:
                        is_started = True
                        pg.mixer.music.stop()
                        pg.mixer.music.load("fig/sound/game_bgm.mp3")
                        pg.mixer.music.play(-1)
                elif game_over:
                    if event.key == pg.K_r:
                        main()
                        return

        # プレイ中の更新処理
        if is_started and not game_over:
            current_speed = 8 + score.score_value // 10

            if tmr % 60 == 0:
                obstacles.add(Obstacle(current_speed))

            if tmr % 80 == 0:
                coins.add(Coin(current_speed, coin_img))
```

```python

            bird.update(key_lst)
            obstacles.update()
            coins.update()

            for coin in pg.sprite.spritecollide(bird, coins, True):
                se_coin.play()
                score.coin_value += 1

            if pg.sprite.spritecollide(bird, obstacles, False):
                game_over = True
                pg.mixer.music.stop()
                se_gameover.play()

            for obstacle in obstacles:
                if obstacle.rect.x < -10 and not hasattr(obstacle, "scored"):
                    score.score_value += 1
                    obstacle.scored = True

        # =================================
        # 描画処理
        # =================================

        if game_over:
            # ゲームオーバー時は他のオブジェクトの描画を一切行わない
            # 1. 明るめに調整された「died_kokaton.png」の背景を一番前に描画
            screen.blit(bg_img_go, (0, 0))

            # 2. その上からゲームオーバー関連の文字を重ねる
            over_str = "GAME OVER"
            score_str = f"Final Score : {score.score_value}"
            coin_str = f"Coins : {score.coin_value}"
            retry_str = "Press R to Retry"

            over_w = go_title_font.size(over_str)[0]
            score_w = go_score_font.size(score_str)[0]
            coin_w = go_score_font.size(coin_str)[0]
            retry_w = msg_font.size(retry_str)[0]

            draw_text_with_shadow(screen, over_str, go_title_font, RED, BLACK, WIDTH//2 - over_w//2, HEIGHT//
            draw_text_with_shadow(screen, score_str, go_score_font, YELLOW, BLACK, WIDTH//2 - score_w//2, HEI
            draw_text_with_shadow(screen, coin_str, go_score_font, YELLOW, BLACK, WIDTH//2 - coin_w//2, HEIGH
            draw_text_with_shadow(screen, retry_str, msg_font, WHITE, BLACK, WIDTH//2 - retry_w//2, HEIGHT//2

        else:
            # 通常時（プレイ中またはスタート画面）の描画
            # 背景のスクロール描画
            bg_scroll_x = tmr % bg_w
            screen.blit(bg_img, (-bg_scroll_x, 0))
            if bg_w - bg_scroll_x < WIDTH:
                screen.blit(bg_img, (bg_w - bg_scroll_x, 0))

            # 地面のスクロール描画
            current_speed = 8 + score.score_value // 10 if is_started else 8
            ground_scroll_x = (tmr * current_speed) % ground_size
            for x in range(-ground_size, WIDTH + ground_size, ground_size):
                screen.blit(ground_tile, (x - ground_scroll_x, GROUND_Y))

            # ゲーム中のオブジェクト描画
```

```
            obstacles.draw(screen)
            coins.draw(screen)
            screen.blit(bird.image, bird.rect)

            # スタート画面の文字描画
            if not is_started:
                title_str = "KOKATON RUN"
                start_str = "Press SPACE to Start"
                title_w = title_font.size(title_str)[0]
                start_w = msg_font.size(start_str)[0]
                draw_text_with_shadow(screen, title_str, title_font, YELLOW, BLACK, WIDTH//2 - title_w//2, HEI
                draw_text_with_shadow(screen, start_str, msg_font, WHITE, BLACK, WIDTH//2 - start_w//2, HEIGHT
            else:
                # プレイ中のスコア表示
                score.update(screen)

        pg.display.update()

        if is_started and not game_over:
            tmr += 1

        clock.tick(60)

if __name__ == "__main__":
    pg.init()
    main()
    pg.quit()
    sys.exit()
```